

RELAZIONE DI PROGETTO: APPRENDIMENTO AUTOMATICO E APPRENDIMENTO PROFONDO

Studente: Gabriele Astolfi

Matricola: 0322600084

Corso di Laurea: Magistrale in Ingegneria Informatica e dell'Intelligenza Artificiale Applicata

1. Introduzione e Analisi del Dataset

Il progetto ha l'obiettivo di sviluppare e confrontare modelli di Machine Learning e Deep Learning per l'identificazione automatica di anomalie di bordo, partendo da dati di telemetria reali. Il dataset che ho scelto è lo **Statlog (Shuttle) Dataset** del repository UCI.

I dati contengono 9 feature numeriche che descrivono le letture dei sensori di uno Space Shuttle della NASA durante le missioni di volo. Il target prevede una classificazione multiclasse suddivisa in 7 stati operativi: la classe 1 rappresenta il funzionamento nominale (volo corretto), mentre le classi da 2 a 7 identificano anomalie di sistema (es. problemi termici o di valvole). L'applicazione di questi modelli simula i tipici sistemi di controllo di volo e monitoraggio delle *Mission Operations* in ambito aerospaziale.

Per garantire la piena compatibilità locale in ambiente Windows, ho focalizzato l'analisi sul sottoinsieme scompattato nativo composto da 14.500 campioni, mantenendo l'integrità di tutte le feature numeriche.

2. Partizionamento dei Dati e Preprocessing

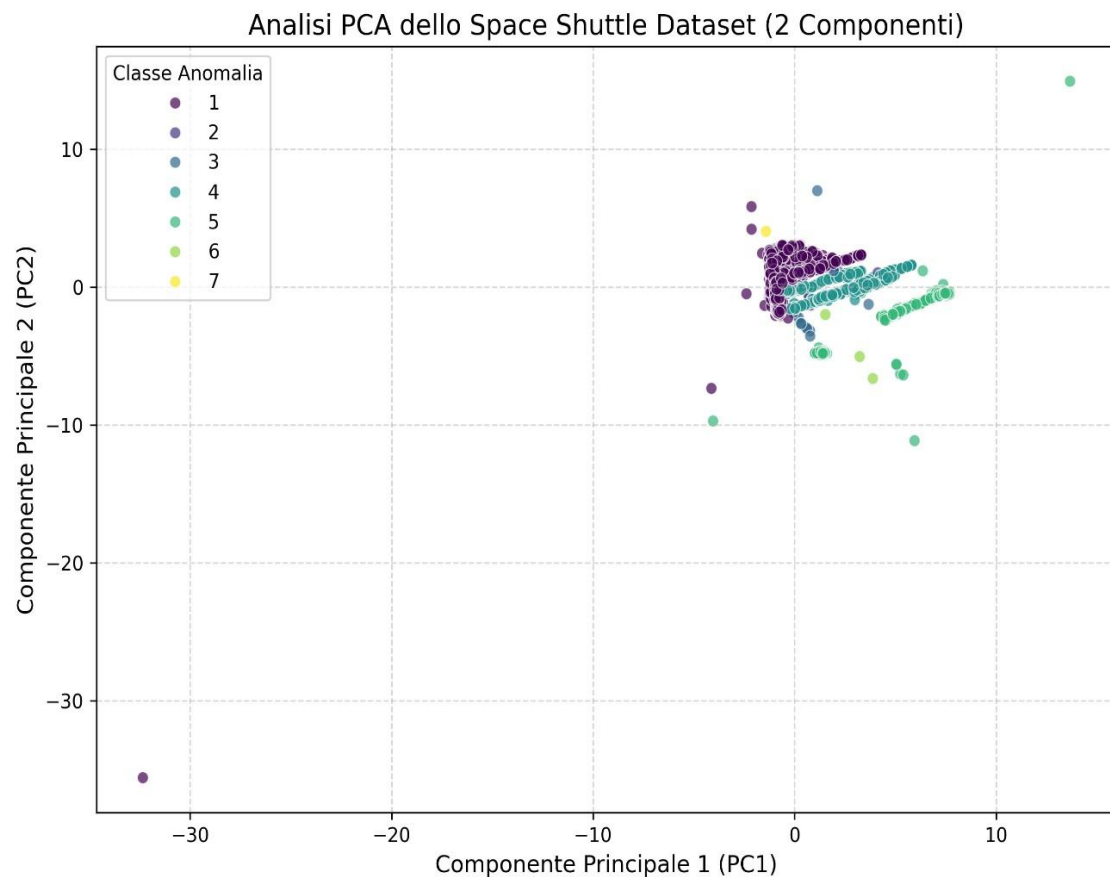
Per evitare *Data Leakage* e garantire la coerenza metodologica nel confronto tra i diversi algoritmi, ho suddiviso il dataset in tre subset distinti:

- Training Set (70%): 10.150 campioni utilizzati per l'addestramento dei modelli.
- Validation Set (15%): 2.175 campioni dedicati al monitoraggio della convergenza della rete neurale.
- Test Set (15%): 2.175 campioni riservati esclusivamente alla valutazione finale delle metriche.

Prima dell'applicazione degli algoritmi, ho eseguito la standardizzazione dei dati tramite *StandardScaler*. Questo passaggio centra la media sullo zero e fissa la varianza a uno per ogni sensore, un'operazione che serve ad evitare sbilanciamenti nei modelli basati sulle distanze.

3. Task 1: Riduzione della Dimensionalità (PCA)

Ho implementato l'algoritmo PCA (Principal Component Analysis) riducendo lo spazio delle feature a 2 componenti principali. Le prime due componenti descrivono il 52.77% della varianza totale dei dati dei sensori. Dal grafico della PCA emerge chiaramente la concentrazione della classe nominale (classe 1) in un cluster denso e centrale, mentre le anomalie operative (classi 3, 4, 5) si proiettano invece lungo traiettorie geometriche separate rispetto al funzionamento normale. Questa disposizione conferma visivamente l'alto livello di separabilità del dataset.



4. Task 2 & 3: Modelli di Classificazione e Metriche

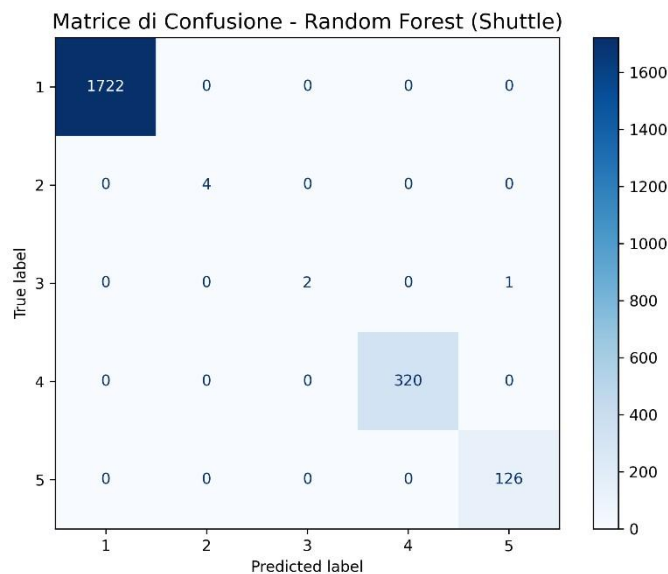
I modelli scelti per la classificazione sono: Logistic Regression (utilizzato come baseline lineare), Random Forest (modello ensemble basato su alberi) e K-Nearest Neighbors (KNN) (modello geometrico basato sulle distanze). Dato il forte sbilanciamento delle classi nel dataset dello Shuttle, le metriche di Precision, Recall e F-Measure sono state calcolate applicando la media pesata (average='weighted').

Tabella Comparativa dei Risultati (Test Set):

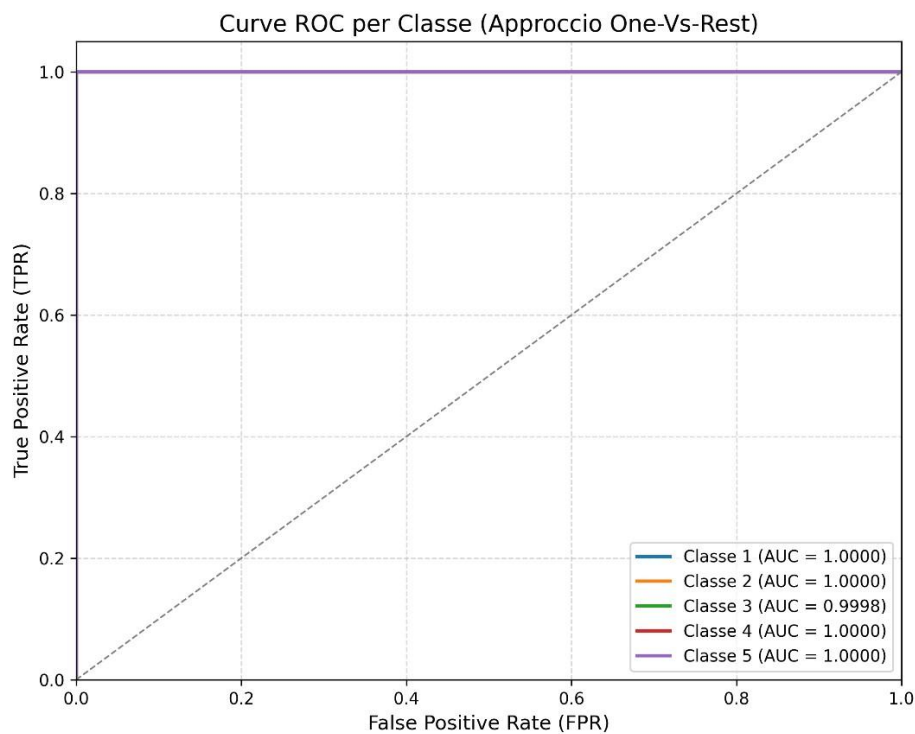
Modello	Accuracy	Precision	Recall	F-Measure
Logistic Regression	96.97%	96.61%	96.97%	96.77%
Random Forest	99.95%	99.95%	99.95%	99.95%
K-Nearest Neighbors	99.68%	99.54%	99.68%	99.61%

5. Task 4: Valutazione Grafica (Confusion Matrix e Curve ROC)

La Matrice di Confusione del modello Random Forest mostra una precisione elevata nel riconoscere le anomalie, registrando un solo errore di classificazione (un campione di classe 3 classificato erroneamente come classe 5) su 2.175 totali.



Le Curve ROC calcolate con la strategia *One-Vs-Rest* confermano la robustezza del classificatore, mostrando un'area sotto la curva (AUC) pari a 1.0000 per la quasi totalità delle classi analizzate.



6. Task 5: Approccio di Deep Learning e Conclusioni

Come richiesto dal punto 5.a della traccia, ho implementato una rete neurale artificiale profonda di tipo MultiLayer Perceptron (MLP). L'architettura prevede due strati nascosti composti rispettivamente da 64 e 32 neuroni, l'utilizzo della funzione di attivazione ReLU e l'ottimizzatore Adam. L'addestramento tramite *Backpropagation* è terminato all'epoca 102 per raggiunta convergenza della funzione di perdita (*Loss* pari a 0.00092). Le metriche sul Test Set hanno registrato un'Accuracy del 99.91% (con Precision, Recall e F-Measure attestate al 99.91%).

7. Conclusioni finali

Il confronto dimostra che sia l'approccio ensemble (Random Forest: 99.95%) sia l'approccio Deep Learning (MLP: 99.91%) riescono a modellare la telemetria dello Shuttle con un livello di accuratezza elevato. Tuttavia, il modello Random Forest resta la scelta preferibile perché in termini di efficienza computazionale richiede tempi di addestramento nettamente inferiori rispetto alla rete neurale a parità di performance sulle anomalie. Di seguito è riportato il codice completo suddiviso per celle caricato su GitHub in formato Jupyter Notebook:

```
# =====
# PROGETTO APPRENDIMENTO AUTOMATICO E PROFONDO - SPACE SHUTTLE ANOMALY
# =====
# -----
# CELLA 1: Caricamento e Preprocessing dei Dati
# -----
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
confusion_matrix, ConfusionMatrixDisplay,
roc_curve, auc
nomi_colonne = ['Sensore_1', 'Sensore_2', 'Sensore_3', 'Sensore_4', 'Sensore_5', 'Sensore_6', 'Sensore_7',
'Sensore_8', 'Sensore_9', 'Classe']
dataset_shuttle = pd.read_csv('shuttle.tst', sep=r'\s+', names=nomi_colonne)
dataset_shuttle.to_csv('shuttle_completo.csv', index=False)
# -----
# CELLA 2: Partizionamento dei Dati, Standardizzazione e PCA
# -----
df = pd.read_csv('shuttle_completo.csv')
X = df.drop(columns=['Classe'])
y = df['Classe']
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)
scaler = StandardScaler()
```

```

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
plt.figure(figsize=(10, 7))
sns.scatterplot(x=X_train_pca[:, 0], y=X_train_pca[:, 1], hue=y_train, palette='viridis', alpha=0.7, legend='full')
plt.title('Analisi PCA dello Space Shuttle Dataset (2 Componenti)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
# -----
# CELLA 3: Addestramento Modelli Classici e Metriche
# -----
modelli = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5)
}
risultati = []
for nome, modello in modelli.items():
    modello.fit(X_train_scaled, y_train)
    y_pred = modello.predict(X_test_scaled)
    y_proba = modello.predict_proba(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    rec = recall_score(y_test, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
    try:
        auc_val = roc_auc_score(y_test, y_proba, multi_class='ovr', average='weighted')
    except Exception:
        auc_val = np.nan
    risultati.append({"Modello": nome, "Accuracy": f"{acc*100:.2f}%", "Precision": f"{prec*100:.2f}%", "Recall":
f"{rec*100:.2f}%", "F-Measure":
f"{f1*100:.2f}%"})
df_risultati = pd.DataFrame(risultati)
print(df_risultati.to_string(index=False))
# -----
# CELLA 4: Confusion Matrix e Curve ROC
# -----
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)
y_pred = rf_model.predict(X_test_scaled)
y_proba = rf_model.predict_proba(X_test_scaled)
cm = confusion_matrix(y_test, y_pred)
classi_presenti = np.unique(y_test)
fig, ax = plt.subplots(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=classi_presenti)
disp.plot(cmap='Blues', ax=ax, values_format='d')
plt.show()

```

```

y_test_bin = label_binarize(y_test, classes=classi_presenti)
n_classes = y_test_bin.shape[1]
plt.figure(figsize=(9, 7))
for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_proba[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f'Classe {classi_presenti[i]} (AUC = {roc_auc:.4f})')
    plt.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')
    plt.legend(loc="lower right")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
# -----
# CELLA 5: Approccio Deep Learning (MLP)
# -----
mlp = MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu', solver='adam', max_iter=500,
random_state=42, verbose=True)
mlp.fit(X_train_scaled, y_train)
y_pred_mlp = mlp.predict(X_test_scaled)
print(f"Accuracy DL: {accuracy_score(y_test, y_pred_mlp)*100:.2f}%")
print(f"Precision DL: {precision_score(y_test, y_pred_mlp, average='weighted', zero_division=0)*100:.2f}%")
print(f"Recall DL: {recall_score(y_test, y_pred_mlp, average='weighted', zero_division=0)*100:.2f}%")
print(f"F-Measure DL: {f1_score(y_test, y_pred_mlp, average='weighted', zero_division=0)*100:.2f}%")

```